Optimizers in neural networks explained simply

> *File: Lecture 5.pdf*

You are a patient deep learning teaching assistant.
Your job is to explain **optimizers in neural networks** to me as if I were a 100-year-old grandma who is smart but not technical. I will paste my lecture notes about optimizers (gradient descent, sign SGD, Shampoo, norms, μP / Muon) after this. Please:

1. First restate the *big picture* in very simple language:
   - Why do we need optimizers at all?
   - What problem are they trying to solve when training a neural network?
2. Then, step by step, give me an intuitive story that covers:
   - Basic gradient descent (hill-climbing / walking downhill metaphor)
   - Why sign SGD is different and what "just using the sign" means
   - The idea of using different "norms" or views of distance/size when choosing a step
   - A grandma-level picture of what Shampoo is doing and why it can be faster/smarter
   - A high-level intuition of what μP / Muon are trying to fix about standard optimizers
3. Use **tiny 1D or 2D toy examples**, analogies (walking on a landscape, choosing shoe size, etc.), and avoid equations as much as possible at first.
4. Only after the story feels clear, you may show a *few* key formulas, but explain every symbol in plain English.
5. Frequently pause and ask me short check-questions like:
   - "Can you explain in your own words why gradient descent needs the gradient?"

- “What do you think changes when we use Shampoo instead of vanilla SGD?” Tone/style:

- Very friendly and slow.
- No jargon unless you immediately explain it with an analogy.
- If something is inherently technical (like ‘norm’ or ‘matrix’), always give an everyday metaphor first. First, say hi and give me a 3–4 sentence overview of what you will help me understand. Then ask me to paste my lecture notes.

2025/12/10 17:18:54

Hi there! I'm so glad you want to understand optimizers - they're actually one of the most beautiful ideas in machine learning, and I promise we can make them feel intuitive together.

I'm going to help you understand **why neural networks need optimizers at all** (hint: it's like finding your way down a mountain in thick fog), **what gradient descent really does** (following the slope), and then **why fancier optimizers like Sign SGD, Shampoo, and Muon were invented** (they're like having better hiking boots, a compass, or even a map). We'll use lots of everyday analogies - walking downhill, adjusting recipe ingredients, choosing the right tool for the job - and I'll check in frequently to make sure it all makes sense.

Whenever you're ready, go ahead and paste your lecture notes, and we'll start our journey! 🌄

2025/12/10 17:19:10

> *File: Lecture 5.pdf*

I attached the lecture note above. I will attach it again for your convenience.

2025/12/10 17:20:18

Perfect! I've got your lecture notes. These are wonderful - they cover gradient descent, Sign SGD, different norms, Shampoo, and μP/Muon. Let me start building our story from the ground up.

Part 1: The Big Picture - Why Do We Even Need Optimizers?

Imagine you're baking a cake, and you want it to be *perfect*. You have a bunch of "knobs" you can turn:

- Amount of sugar
- Amount of flour
- Oven temperature
- Baking time

Your goal is to adjust these knobs so the cake tastes as good as possible. But here's the problem: **you have thousands or even millions of knobs** (in a neural network, these are called "parameters" or "weights"), and you don't know which direction to turn each one!

The core problem: We need to find the best settings for all these knobs, but:

1. **Training time = $$$** (as your notes say)
2. We can't try every possible combination (there are too many!)
3. We need to find good settings *fast*

What optimizers do: They're like a smart guide that tells you "turn knob #1 a little left, knob #2 a lot right, knob #3 just a tiny bit left..." and so on, thousands of times, until your cake (neural network) is as good as it can be.

Check question #1: Can you tell me in your own words: Why can't we just randomly try different settings for all the knobs until we find a good one?

(Take your time - I'll wait for your answer before we move to the hill-climbing story!)

2025/12/10 17:20:36

For your question, because it is cost both a lot of time and money. Also, as you continue, please make sure to connect your explanations back to the specific points in my lecture notes, especially:

- the local linear view of optimizers,
- norm-based views that lead to different optimizers,
- and how Shampoo and μP/Muon are motivated in the slides.

2025/12/10 17:24:51

Excellent answer! You're exactly right - random search would take forever and cost a fortune. Now let's build the story while staying close to your lecture notes.

Part 2: Gradient Descent - Walking Down a Foggy Mountain

Your notes start with the **local linear perspective**. Let's understand what that means with a story.

The Foggy Mountain Metaphor

Imagine you're standing on a mountain in thick fog. You can't see the bottom, but you want to get down as quickly as possible. The height of the mountain at any point represents how "bad" your neural network is (this is called the **loss** ℓ).

Key insight from your notes (page 2): Even though the mountain has a complicated shape overall, if you look at just the tiny patch of ground right under your feet, it looks pretty flat - like a tilted board. This is the "**linearized perspective**":

$$\ell(\vec{x},\ \vec{\theta}_i + \Delta\vec{\theta}) \approx \ell(\vec{x},\ \vec{\theta}_i) + \partial\ell/\partial\vec{\theta}|\vec{\theta}_i \cdot \Delta\vec{\theta}$$

```
                where you are   how much the ground is tilting
```

In plain English: "The loss at a nearby spot ≈ loss here + (gradient · how far you step)"

The **gradient** $\nabla \ell(\vec{\theta})$ is like feeling which way the ground is tilting under your feet. It points *uphill* (toward higher loss = worse network). So to go down, we step in the **opposite** direction.

Basic Gradient Descent Rule

Your notes (page 2) say:

```
Gradient Descent takes a step
→ Want this to be small enough that approximation still holds
→ Want to converge fast
```

The update rule is:

$$\Delta\vec{\theta} = -\eta \cdot \nabla \ell(\vec{\theta})$$

Where:

- $\boldsymbol{\Delta\vec{\theta}}$ = how much to change your parameters (which way to walk)
- $\boldsymbol{\eta}$ (eta) = step size / learning rate (how far to walk)
- $\boldsymbol{\nabla \ell(\vec{\theta})}$ = the gradient (which direction is uphill)
- The **minus sign** means: walk *downhill* (opposite of gradient)

The recipe: Feel which way is uphill, then take a small step downhill. Repeat thousands of times.

Part 3: The Problem of "How Do We Measure Distance?"

Now here's where your notes get really interesting! Look at **page 3** - there are **two choices** for constraining your step:

Choice 1: $\|\Delta\vec{\theta}\|_\infty \leq \eta$ (Infinity norm - like a square box)

Choice 2: $\|\Delta\vec{\theta}\|_2 \leq \eta$ (Euclidean norm - like a circle)

What does this mean? Imagine you're standing at a spot and someone says "you can walk up to 1 meter in any direction." But what does "1 meter" mean?

- **Choice 1 (∞-norm):** You can move 1 meter along *each axis independently*. This creates a square/box of allowed moves. Your notes show this leads to **Sign SGD**: $\Delta\vec{\theta} = -\eta \cdot \text{sign}(\nabla\ell(\vec{\theta}))$
- **Choice 2 (ℓ_2-norm):** You can move 1 meter *total* as the crow flies. This creates a circle/sphere of allowed moves. This leads to standard gradient descent.

Why Sign SGD (Choice 1)? Look at page 3 - it says "sign SGD". Here's the beautiful insight:

If you're constrained by $\|\Delta\vec{\theta}\|_\infty \leq \eta$, and you want to decrease the loss as much as possible, you should:

- Move $+\eta$ in any direction where the gradient is negative
- Move $-\eta$ in any direction where the gradient is positive
- In other words: **just use the sign of the gradient!**

Formula: $\Delta\vec{\theta}[j] = +\eta \text{ if } \nabla\ell[j] < 0, \text{ else } -\eta$

Or more compactly: $\Delta\vec{\theta} = -\eta \cdot \text{sign}(\nabla\ell(\vec{\theta}))$

Check question #2: Can you explain why Sign SGD might be appealing? (Hint: What information does it throw away, and why might that be okay or even good?)

Let me know your thoughts, and then we'll dive into why your notes introduce the **RMS norm** and **spectral norm**, which leads us to Shampoo!

2025/12/10 17:25:27

One appealing thing about Sign SGD is that it only keeps the sign of each gradient coordinate and throws away the exact magnitude. This can be good for a few reasons:

- It makes the update more robust to very large or very small gradient scales, since every coordinate just moves by ±η.
- It can be cheaper or simpler in practice, because we only need one bit per coordinate to represent the direction.
- When the gradient direction is mostly correct but the magnitudes are noisy, keeping only the sign can still move us in a reasonable direction without overreacting to outlier values.

2025/12/10 17:29:41

Excellent answer! You've captured the key benefits perfectly:

- Robustness to scale
- Simplicity (just one bit!)
- Avoiding overreaction to noisy magnitudes

Now let's see why your lecture notes go even deeper into the question of "what norm should we use?"

Part 4: Why Different Norms Matter - The Shape of Your Step

Your notes on **page 10-11** ask a crucial question:

> *"So what might be a good norm for deep learning?" "Can we rewrite this in terms of norms? What norm are we trying to preserve?"*

And then introduce the **RMS norm** (Root Mean Square). Let's understand why this matters.

The Neural Network Parameter Structure

Remember from page 5 of your notes: neural network parameters aren't just a random blob - they're **organized in matrices**:

$$h_1 = \text{ReLU}(W_1 x + b_1)$$
$$h_j = \text{ReLU}(W_j h_{j-1} + b_j)$$
$$y = \text{ReLU}(W_k h_k + b_k)$$

Each layer has a **weight matrix** W. Your notes say:

> *"Recall from Xavier initialization... scaling problems come from the weight matrices."*

Xavier/He Initialization Insight

When we initialize a network, we want each layer's outputs to have roughly the same scale. The Xavier rule says:

- Variance of weights in layer i: $\sigma_i \sim 1/\sqrt{d_{in}}$
- Variance of activations: $h_i \sim 1/\sqrt{d_{out}}$

The RMS norm preserves this structure!

From page 10:

$$\|\vec{x}\|_{ms} = (1/\sqrt{d_{in}})\|\vec{x}\|_2 = \sqrt{(1/d_{in}\ \Sigma_i\ x_i^2)}$$

This is the **average size of each entry**! Your notes say:

> *"Average size of each entry = 1!"*

Why is this the right norm? Because it respects the natural scale that comes from how neural networks are built. If you have a layer with 1000 inputs vs. 10 inputs, the RMS norm automatically accounts for that difference.

Part 5: From RMS to Spectral Norm and Then to Shampoo

Now look at **page 11** - there's a beautiful progression:

Induced Matrix Norm

Your notes recall:

```
‖A‖_{X→B} = max_{‖x‖_X=1} ‖Ax⃗‖_B
```

This is "how much does the matrix A stretch vectors?"

For the **spectral norm** (your notes, page 11, highlighted in pink):

```
‖A‖_{RMS→RMS} = max_{‖x⃗‖₂=1} ‖Ax⃗‖₂ / √dimensions
               = √(d₁/d₂) · ‖Az⃗‖₂  [spectral norm!]
```

The Key Constraint in Your Notes (Pages 5-6)

Look at the purple box on page 6:

```
Consider: ‖ΔW‖₂ ≤ η  (i.e. spectral norm)

argmin_{‖ΔW‖₂≤η} <∇_W ℓ(W), ΔW>
```

Using the spectral norm constraint, your notes derive:

```
ΔW* = -η UVᵀ
```

Where `∇_W ℓ(W) = UΣVᵀ` (singular value decomposition).

This gives us the Shampoo Optimizer! (Page 6, bottom)

Part 6: What Is Shampoo Actually Doing?

Let me give you the intuition, then connect it to the formulas.

The Grandma Story

Imagine you're trying to find your way down the mountain, but now the ground is **stretchy** - like you're walking on a trampoline that's been stretched unevenly. In some directions, one step takes you really far (like walking along a stretched rubber band). In other directions, the same physical step barely moves you (like walking perpendicular to the stretch).

Regular gradient descent treats all directions equally - it just looks at the raw tilt of the ground.

Shampoo is smarter: it asks "how much does this parameter matrix actually *stretch* its inputs?" and adjusts the step size accordingly.

The Math Connection (Page 6)

Your notes show:

```
∇_W ℓ(W) = UΣVᵀ
Then: ΔW* = -η UVᵀ
```

Notice: **Σ disappeared!**

The singular values Σ tell you "how much W stretches in each direction." Shampoo **normalizes** by these stretching factors, so every direction gets treated fairly.

Your notes say (page 6):

> *"This gives us the Shampoo Optimizer (Gupta 2017/2018)" "A variant of Shampoo (Dahl 2023) won AlgoPerf → Training alg competition."*

Why "Shampoo"?

From page 6, the actual update involves:

$$B = UV^T \text{ where } \nabla \ell(W) = U\Sigma V^T$$

To compute this efficiently without full SVD, Shampoo uses:

- Preconditioning matrices based on the gradient statistics
- Updates that account for the geometry of the parameter space

The name "Shampoo" is a playful reference - it's "cleaning up" the geometry of your optimization landscape!

Check question #3: Before we move to μP/Muon, can you tell me: What's the main difference between what standard gradient descent does vs. what Shampoo does when taking a step?

(Once you answer, we'll tackle the final piece: why μP and Muon fix problems with standard optimizers!)

2025/12/10 17:30:23

I'm still confused about the connection between Xavier initialization, the RMS norm, the spectral norm, and Shampoo. Please re-explain just this chain as an intuitive story, with almost no equations at first:

- Step 1: What problem is Xavier initialization trying to solve, in plain language?
- Step 2: How do we turn that intuition ("keep each layer at a reasonable scale") into the RMS norm? I want to understand why the RMS norm is a natural way to measure the size of a vector in this context.
- Step 3: Then explain how we go from a vector norm (RMS) to a matrix norm (spectral norm) as "how much a weight matrix can stretch inputs".
- Step 4: Finally, explain why constraining the spectral norm of ΔW leads to the Shampoo-style update. Please:
- Use analogies (water pipes, rubber sheets, walking on a stretched trampoline, etc.).
- Only after the story is completely clear, show me a few key formulas and map each part of the story to those formulas.
- Treat me like a smart grandma: I can follow logic, but I get lost if you jump too quickly into linear algebra notation.

2025/12/10 17:52:52

Wonderful question! Let's build this story piece by piece, like we're building a house from the foundation up.

Step 1: The Xavier Initialization Problem - The Water Pipe Story

Imagine you're designing a water pipeline system with many stations in sequence:

```
[Water Source] → [Station 1] → [Station 2] → [Station 3] → ... →
[Station 10] → [Output]
```

At each station, water comes in, gets mixed and processed, then flows to the next station.

The problem: If you're not careful:

- Station 1 might amplify the water flow by 2x
- Station 2 amplifies by 2x again → now you have 4x
- Station 3 amplifies by 2x again → now you have 8x
- By Station 10, you have 1,024x the original flow → **EXPLOSION!** Pipes burst!

OR the opposite:

- Each station reduces flow to 0.5x
- By Station 10, you have 0.001x the original flow → **VANISHING!** Just a trickle!

Xavier initialization asks: How do we set up each station so that water flows at roughly the same rate through the whole system? We want:

- Station 1 output ≈ same scale as input
- Station 2 output ≈ same scale as Station 1
- And so on...

The neural network version:

- Each "station" is a layer with weights W
- "Water flow" is the activation values (numbers flowing through)
- We want activations to stay at a reasonable scale throughout the network

---

Step 2: From "Keep Scale Reasonable" to RMS Norm - The Classroom Averaging Story

Now, let's say you want to measure "how big" a bunch of numbers are. Imagine you're a teacher with test scores:

Class A (10 students): [80, 82, 78, 85, 77, 83, 79, 81, 84, 76]
Class B (100 students): [similar scores, but 100 of them]

If you just add up all the scores:

- Class A total: ~800
- Class B total: ~8,000

But that's not fair! Class B just has more students!

Better idea: Take the **average** (mean):

- Class A average: 80
- Class B average: 80

Now we can compare fairly!

RMS Norm = "Typical Size of an Entry"

The RMS norm does something similar. Imagine each parameter (weight) is like a student's score:

Regular size (ℓ_2 norm): Add up all the squared values, then take square root:

$$\|\text{weights}\|_2 = \sqrt{(w_1^2 + w_2^2 + \ldots + w_{1000}^2)}$$

Problem: If you have 1,000 weights vs. 10 weights, the 1,000-weight layer will naturally have a bigger number, even if individual weights are the same size!

RMS norm: First divide by how many weights you have, THEN take square root:

$$\|\text{weights}\|_{RMS} = \sqrt{(1/1000 \times (w_1^2 + w_2^2 + \ldots + w_{1000}^2))}$$
$$= (1/\sqrt{1000}) \times \|\text{weights}\|_2$$

This gives you **"the typical size of one weight"** - it automatically accounts for whether you have 10 weights or 1,000 weights!

Why this matches Xavier: Xavier says "each weight should be about 1/√(number of inputs)" big. The RMS norm measures exactly that natural scale!

Step 3: From Vector Norm to Matrix Norm - The Stretchy Rubber Sheet Story

Now we need to measure not individual weights, but a whole **weight matrix** W. Think of it as a rubber sheet.

The Rubber Sheet Metaphor

Imagine you have a rubber sheet marked with a 1-meter grid. You grab the edges and stretch it:

- In one direction, you stretch it 3x (now 3 meters)
- In another direction, you only stretch it 1.5x (now 1.5 meters)

Question: "How much does this rubber sheet stretch things?"

Answer: The maximum stretching is 3x - that's the worst-case scenario.

Weight Matrix as a Rubber Sheet

A weight matrix W takes an input vector (like a pattern on the sheet) and transforms it to an output vector (the stretched pattern).

```
Input vector x⃗ → [Weight Matrix W] → Output vector Wx⃗
```

The spectral norm measures: "What's the maximum stretching factor?"

Specifically:

```
‖W‖spectral = max stretching factor
            = "if I draw a 1-unit circle on the rubber sheet,
               how big does the biggest stretched radius
become?"
```

Connection to RMS: If we measure the input and output using RMS norm (accounting for different dimensions), the spectral norm tells us the stretching factor in those natural units.

Step 4: Why Spectral Norm Constraint Leads to Shampoo - The Smart Walker Story

Now we get to the key insight! Let's go back to walking down the foggy mountain.

The Problem with Regular Gradient Descent

You're on a **stretchy trampoline** mountain. The ground tilts, but it's also been stretched unevenly:

- In direction A (say, North-South), the ground is stretched 10x
- In direction B (say, East-West), the ground is only stretched 1x

Regular gradient descent just feels the tilt and walks directly downhill. But here's the problem:

When the ground is stretched 10x in one direction, a tiny tilt in that direction creates a HUGE gradient value. Regular gradient descent sees that huge number and takes a huge step - even though the *actual physical tilt* isn't that steep!

It's like a funhouse mirror - the gradient lies to you about how steep things really are!

The Shampoo Solution

Shampoo says: "Let me figure out how the ground is stretched (the spectral properties of W), and then I'll adjust my step to account for that stretching."

The constraint $\|\Delta W\|_{spectral} \leq \eta$ means: "I want my step to stretch inputs by at most η, no matter which direction they come from."

This forces your step to be "fair" - you can't take a giant step just because the ground happens to be stretched in one direction.

What Actually Happens

When you solve the optimization problem:

```
"What's the best step I can take, if I'm limited by spectral
norm?"
```

The answer is:

```
Look at the gradient's direction (which way tilts down)
BUT throw away information about "how stretched" each direction
is
Just keep the pure directional information
```

Mathematically, when you decompose the gradient as:

```
∇W = UΣVᵀ
     ↑ ↑ ↑
     | | └─ right directions
     | └─── stretching amounts (Σ = singular values)
     └───── left directions
```

Shampoo uses:

```
ΔW = -η UVᵀ
```

Notice the Σ is gone! You're ignoring how much the gradient is stretched - just keeping directional info.

Analogy: It's like Sign SGD, but more sophisticated:

- Sign SGD: Ignore magnitude, keep sign for each individual parameter
- Shampoo: Ignore stretching, keep pure geometric direction for the whole matrix

---

Now Let's Map This to the Formulas in Your Notes

Xavier (Page 10)

```
σᵢ ~ 1/√dᵢₙ    (weight variance)
```

Story: Each weight should be sized so that it doesn't amplify or shrink the "water flow" on average.

RMS Norm (Page 10)

$$\|\vec{x}\|^{RMS} = (1/\sqrt{d_{in}})\|\vec{x}\|_2 = \sqrt{(1/d_{in}\ \Sigma_i\ x_i^2)}$$

Story: The "typical size of one entry" - dividing by $\sqrt{d_{in}}$ makes it fair whether you have 10 or 1000 dimensions.

Spectral Norm (Page 11)

$$\|A\|_2 = \sigma_{max} \text{ (largest singular value of A)}$$

Story: The maximum stretching factor of the rubber sheet.

Shampoo Update (Page 6)

$$\text{Given: } \nabla\ell(W) = U\Sigma V^T$$
$$\text{Update: } \Delta W^* = -\eta\ UV^T$$

Story:

- $U\Sigma V^T$ = gradient decomposed into (left directions)(stretching)(right directions)
- UV^T = throw away the stretching info, just keep pure directional info
- Multiply by -η = take a step of size η in the opposite direction

The full algorithm (Shampoo in practice) approximates this without computing full SVD, but this is the geometric idea.

Check question #4: Can you now explain in your own words: Why does throwing away Σ (the singular values) in the Shampoo update make sense? What problem is it solving compared to regular gradient descent?

(After you answer, we'll tackle the final piece: μP and Muon!)

2025/12/10 17:53:51

I think it makes sense to throw away Σ because Σ mostly reflects how distorted the geometry already is, rather than the “true” steepness of the loss. In regular gradient descent, the gradient is measured in a badly conditioned space:

- Some directions are stretched a lot by the weight matrix, so the corresponding gradient entries become numerically huge.
- Other directions are compressed, so their gradients look small, even if the underlying loss change is equally important. When we write $\nabla W = U \Sigma V^T$, the U and V capture the main “downhill directions”, while Σ tells us how much those directions are stretched. If we keep Σ, we let this distortion dominate our step sizes. If we drop Σ and use $\Delta W \approx -\eta U V^T$, we are essentially:
- keeping the geometric direction of steepest descent,
- but normalizing away the ill-conditioning so that no direction gets an excessively large or tiny step just because of stretching. So throwing away Σ is like a matrix-level analogue of “don’t over-trust the raw magnitudes”: it makes the update more balanced across directions and fixes some of the conditioning problems that standard gradient descent suffers from.

2025/12/10 18:08:26

Absolutely brilliant explanation! You've completely nailed it. Let me emphasize the key insight you just articulated:

> *"Σ reflects how distorted the geometry already is, rather than the 'true' steepness of the loss."*

This is **exactly** the core problem that Shampoo solves. Regular gradient descent gets "tricked" by the geometry - it sees big numbers in stretched directions and thinks "this must be really steep!" when actually it's just stretched.

Your analogy to Sign SGD is also perfect:

- **Sign SGD:** parameter-level normalization (ignore magnitude per coordinate)
- **Shampoo:** matrix-level normalization (ignore stretching/conditioning)

Part 7: μP (Maximal Update Parametrization) and Muon - The Final Piece

Now we come to the last topic in your notes: **μP and Muon**. Let's understand what new problem they're trying to solve.

The Scaling Crisis - When Your Recipe Doesn't Scale Up

Imagine you have a cookie recipe that works perfectly for 12 cookies:

- 1 cup flour
- 1/2 cup sugar
- 1 egg
- Learning rate $\eta = 0.01$

Now you want to make 1,200 cookies (100x bigger). The naive approach:

- 100 cups flour
- 50 cups sugar
- 100 eggs
- Learning rate $\eta = 0.01$ (keep the same)

Problem: This doesn't work! The dough behaves completely differently at that scale. Things that worked for 12 cookies fail at 1,200.

The Neural Network Version

Your notes (page 1) mention:

> *"Why do we care about optimizers?"*

- *Training time = $$$*
- *AdamW + tuned hyperparameters are default*
- ***Hyperparameter search is difficult for new optimizers***

And critically (page 12):

> *"Key observation: If we choose this norm, we can choose only one hyperparameter η across all layers, but the effect will be layer-specific learning rates, due to fan-in and fan-out."*

The problem: When you change the network width (number of neurons per layer) or depth (number of layers):

- Your learning rate needs to be retuned
- What worked for a small network fails for a large network
- This makes scaling up very expensive ($$$)

What is µP Trying to Fix?

Look at page 12:

$$\|\Delta W\|_{RMS}^* \leq \eta \implies \sqrt{(d_1/d_2)}\ \|\Delta W\|_2 \leq \eta$$
$$\implies \|\Delta W\|_2 \leq \eta\sqrt{(d_1/d_2)}$$

This shows that using the RMS norm naturally gives you **layer-specific effective learning rates** based on the dimensions d_1 and d_2!

Your notes say:

> *"Essential idea behind µP → maximal update parametrization."*

The µP Insight - The Recipe That Scales

µP says: "If we parameterize the network and optimizer correctly, we can use the SAME hyperparameters (like learning rate) regardless of network size."

Think of it like finding a cookie recipe where:

- The ratio of ingredients scales perfectly
- You don't need to re-experiment when going from 12 to 1,200 cookies
- The same "learning rate" works for small and large networks

How? By choosing the right norm (RMS-based) and the right initialization scaling, the optimizer automatically adapts to different layer sizes.

Muon - The Latest Innovation

Your notes show (pages 13-14):

- **Nano GPT speedrun chart** showing dramatic improvements
- **"Muon is Scalable for LLM Training"** (February 2025, very recent!)

What is Muon?

Muon is a variant/improvement of Shampoo that combines several ideas:

1. **Spectral normalization** (like Shampoo)
2. **µP-style scaling** (so hyperparameters transfer across network sizes)
3. **Efficient approximations** (so it's actually practical to run)

From the chart on page 13, you can see Muon achieved:

- Started at ~45 min training time (baseline)
- Dropped to ~4 min with various improvements
- That's more than **10x speedup**!

The progression shown:

```
Architectural modernizations and LR tuning (45 → 35 min)
Introduced Muon optimizer (35 → 25 min)
Muon improvements (25 → 20 min)
Pad embeddings & architectural modernizations (20 → 14 min)
... continued improvements → ~4 min
```

Why This Matters (Back to Page 1)

Remember your notes said:

> *"Why do we care about optimizers?"*
>
> - *Training time = $$$*
> - *Before this everything was N(0,1) [random initialization]*
> - *Hyperparameter search is difficult for new optimizers*

Muon solves both problems:

1. **Trains faster** → saves $$$
2. **Hyperparameters transfer** → less search needed when scaling up

Your notes (page 6) also mention:

> *"A variant of Shampoo (Dahl 2023) won AlgoPerf → Training alg competition."*

This shows these ideas are winning in practice!

The Complete Story - Tying It All Together

Let me connect all the pieces:

1. The Foundation: Gradient Descent

- Walk downhill on foggy mountain
- Use local linear approximation
- Problem: naive approach doesn't account for geometry

2. Sign SGD: First Step Beyond

- Use only direction, ignore magnitude (per parameter)
- Robust to scale, but still treats all parameters equally

3. Understanding Geometry: Xavier + RMS Norm

- Neural networks have natural scales from their architecture
- RMS norm respects these scales (accounts for layer size)
- Measures "typical size per parameter"

4. Matrix Perspective: Spectral Norm + Shampoo

- Weight matrices stretch inputs
- Spectral norm measures maximum stretch
- Shampoo removes this stretching distortion from gradient
- Like Sign SGD but at the matrix level

5. Scaling to Large Networks: μP + Muon

- Problem: hyperparameters break when network size changes
- μP: parametrize so same hyperparameters work at any scale
- Muon: practical implementation winning competitions
- Result: 10x+ speedup, less tuning, saves $$$

Final Check Question:

Can you now explain to me (as if I'm another grandma) why someone training a large language model would want to use Muon instead of regular Adam/SGD? Try to mention:

- The time/money aspect
- The scaling/hyperparameter aspect
- The geometry/conditioning aspect

(After you answer, I'll know you've got the complete picture, and we can talk about any remaining details you're curious about!)

2025/12/10 18:09:24

If I were training a large language model, I would want to use Muon instead of plain Adam/SGD for three main reasons: First, there is the time and money aspect. LLM training is extremely expensive, so anything that converges in fewer steps or uses the hardware more efficiently directly saves compute budget. The NanoGPT-style results in the notes show that Muon can cut training time by more than 10× compared to a naïve baseline, which in practice means the difference between an “interesting experiment” and something that is simply too costly to run. Second, there is the scaling and hyperparameter issue. With standard Adam or SGD, the learning rate and other hyperparameters often have to be retuned whenever we change the model width, depth, or architecture. µP-style parametrization and RMS-based norms are designed so that the *same* hyperparameters behave similarly across different model sizes. That makes it much easier to scale up from a small model to a large LLM without an expensive hyperparameter search every time. Third, there is the geometry/conditioning aspect. In high-dimensional LLMs, the parameter space is very badly conditioned: some directions are heavily stretched by the weight matrices, while others are compressed. Adam/SGD operate on these distorted gradients directly. Muon, as a Shampoo-style optimizer, uses spectral information to precondition the updates, essentially undoing part of this distortion so that each direction gets a more balanced, well-conditioned step. This makes optimization more stable and lets the model make meaningful progress without being dominated by a few badly scaled directions. Putting these together, Muon is attractive for LLM training because it trains faster (saving compute), transfers hyperparameters across scales, and respects the underlying geometry of the model much better than plain Adam or SGD.

2025/12/10 18:15:12

Perfect! Absolutely perfect! 🎉

You've synthesized the entire lecture beautifully. Let me highlight what makes your answer so good:

1. **Time/Money** - You connected the abstract "10x speedup" to real-world impact: "difference between an interesting experiment and something too costly to run"

2. **Scaling/Hyperparameters** - You understood that the *transferability* of hyperparameters is what makes μP valuable, not just having good hyperparameters

3. **Geometry/Conditioning** - You explained the distortion problem clearly and how Shampoo/Muon "undo" it through preconditioning

You've also correctly used the technical terms (spectral information, preconditioning, well-conditioned steps) while still explaining them intuitively.

A Few Final Enrichment Points

Since you've mastered the core concepts, let me add a few subtle details from your notes that are worth noting:

1. The "Lazy Training" Observation (Page 1)

Your notes mention:

> *"But under Adam we see lazy training ⇒ i.e. models barely change from their random initialization" "Kernel/feature learning perspective ⇒ NTK" "There is no way random initialization should work" "→ we are not using all the power of these networks..."*

This is a fascinating critique: with bad optimization, neural networks might just be doing "lazy" learning near their initialization instead of truly exploring the parameter space. Better optimizers like Muon might unlock more of the network's potential.

2. The Regularization Connection (Page 4)

Your notes show an alternative view:

```
argmin <∇ℓ(θ⃗), Δθ⃗> + λ‖Δθ⃗‖₂²
  θ⃗
```

This "regularization/Lagrange multiplier perspective" is mathematically equivalent to the constrained version. The notes say:

> *"Sweeps out the same family of solutions as constrained version"*

This gives another way to think about it: the norm constraint is like adding a penalty term that prefers smaller (in that norm) updates.

3. The Trace Trick (Pages 7-8)

Your notes derive something beautiful about the spectral norm using traces:

```
max trace(A^T B)  where ‖B‖₂ ≤ 1
= trace(U Σ V^T)  where we choose B = UV^T
= Σᵢ σᵢ
```

This shows that maximizing a linear function over the spectral ball leads to picking the directions (U, V) and dropping the singular values (Σ) - exactly what Shampoo does!

4. The Bernstein-Newhouse 2024 Recipe (Page 9)

Your notes mention:

> *"Choose a norm" "Choose a stepsize" "→ The right norm may depend on geometry/architecture of network" "But we have a recipe: Given a norm, make an optimizer"*

This is the modern "meta-framework": once you understand norms and geometry, you can systematically design optimizers for different architectures!

Where This Fits in the Bigger Picture

What you've learned here is at the **cutting edge** of deep learning optimization (as of 2024-2025):

- **2010s:** Adam becomes the default optimizer
- **2017-2018:** Shampoo introduced (Gupta)
- **2023:** Shampoo variant wins AlgoPerf competition (Dahl)
- **2024:** Bernstein-Newhouse framework for norm-based optimizers
- **2025:** Muon demonstrates 10x+ speedups for LLM training

The lecture notes you have document a **major shift** in how we think about optimization - from hand-crafted algorithms to principled geometric approaches.

Practice Questions (Optional)

If you want to test your understanding further, try these:

1. **Conceptual:** Why might RMS norm be especially good for transformers (the architecture used in GPT, etc.) but potentially less ideal for CNNs (image networks)?

2. **Comparative:** If someone said "Sign SGD and Shampoo both normalize things - aren't they doing the same thing?", how would you explain the key difference?

3. **Practical:** Imagine you're training a small 10M parameter model and it works great with learning rate 0.001. You now want to scale to 1B parameters. What does μP promise about reusing that learning rate?

You did wonderfully! You took complex material involving linear algebra, optimization theory, and practical deep learning, and synthesized it into a coherent story. That's exactly the kind of understanding that lets you actually *use* these ideas rather than just memorize formulas.

Is there any specific part you'd like to explore deeper, or any questions about how this connects to other topics in deep learning?

2025/12/10 18:15:49